

Viva Session: #974663

SYSTEM

(interview start)

Welcome to the viva examination for **2110211 Data Structures**.

Here is the problem scenario for today's session:

Scenario: Cell Detection

A grid consists of n rows and m columns ($1 \leq n, m \leq 100,000$). Initially, all cells are white. There are up to $q = 100,000$ queries processed one by one. The queries are of four types:

1. Paint Cell White: $W \ i \ j$

- **Action:** Change the color of cell (i, j) to **white**.
- **Output:** Print **YES** if the cell was **black** right before this query. Otherwise, print **NO**.

2. Paint Cell Black: $B \ i \ j$

- **Action:** Change the color of cell (i, j) to **black**.
- **Output:** Print **YES** if the cell was **white** right before this query. Otherwise, print **NO**.

3. Check Row Range: $R \ x \ l \ r$

- **Action:** Inspect row x from column l to column r (inclusive).
- **Output:** Print **YES** if **at least one black cell** exists in that column range ($l \leq \text{column} \leq r$). Otherwise, print **NO**.

4. Check Column Range: $C \ x \ l \ r$

- **Action:** Inspect column x from row l to row r (inclusive).
- **Output:** Print **YES** if **at least one black cell** exists in that row range ($l \leq \text{row} \leq r$). Otherwise, print **NO**.

`vector<set<int>>`, `map<pair<int,int>, bool>`, etc.). You must explain both the container choices and the step-by-step logic to process each of the 4 query types.

Come prepared to

1. Name your primary container type(s), specify what keys/elements represent, and describe the exact procedure for handling each of the 4 query types (`W` , `B` , `R` , and `C`).
2. Compare your chosen container and query algorithms against an alternative design (such as a 2D `vector` or a single `set<pair<int,int>>`) in terms of work required per query and memory usage.
3. Explain what goes wrong—or becomes too slow—if we choose a simple 2D grid structure like `vector<vector<bool>>` or `map<pair<int,int>, bool>` given the grid dimensions (`n` , `m` , `q` $\leq 100,000$).

คำอธิบายสถานการณ์ (ภาษาไทย)

ตารางกริดมีขนาด `n` แถว และ `m` คอลัมน์ ($1 \leq n, m \leq 100,000$) เริ่มต้นทุกช่องเป็นสีขาว มีคำถาม/คำสั่งทั้งหมดไม่เกิน `q = 100,000` คำสั่ง ซึ่งประมวลผลทีละคำสั่ง โดยมีคำสั่ง 4 ประเภทดังนี้:

1. เปลี่ยนเป็นสีขาว: `W i j`
 - การทำงาน: เปลี่ยนสีของช่อง (`i` , `j`) เป็น สีขาว
 - ผลลัพธ์: แสดง YES หากช่องนั้นเป็น สีดำ ก่อนหน้าคำสั่งนี้ มิฉะนั้นแสดง NO
2. เปลี่ยนเป็นสีดำ: `B i j`
 - การทำงาน: เปลี่ยนสีของช่อง (`i` , `j`) เป็น สีดำ
 - ผลลัพธ์: แสดง YES หากช่องนั้นเป็น สีขาว ก่อนหน้าคำสั่งนี้ มิฉะนั้นแสดง NO
3. ตรวจสอบช่วงในแถว: `R x l r`
 - การทำงาน: ตรวจสอบแถว `x` ตั้งแต่คอลัมน์ `l` ถึง `r` (รวมขอบ)
 - ผลลัพธ์: แสดง YES หากมี ช่องสีดำอย่างน้อย 1 ช่อง อยู่ในช่วงคอลัมน์ดังกล่าว ($l \leq \text{column} \leq r$) มิฉะนั้นแสดง NO
4. ตรวจสอบช่วงในคอลัมน์: `C x l r`

- ผลลัพธ์: แสดง YES หากมี ช่องสีดำอย่างน้อย 1 ช่อง อยู่ใน ช่วงแถวดังกล่าว ($1 \leq \text{row} \leq r$) มิฉะนั้นแสดง NO

จงตัดสินใจเลือกใช้ **standard container** ของ C++ ในการติดตามช่องสีดำ โดยระบุ ชนิดข้อมูล C++ ที่แน่นอน (เช่น `vector<set<int>>` , `map<pair<int,int>, bool>` ฯลฯ) พร้อมอธิบายขั้นตอนการทำงาน เพื่อประมวลผลคำสั่งทั้ง 4 ประเภท

Let's begin.

What primary C++ container(s) and exact types would you choose to store the state of the grid, what do the stored elements represent, and how would you process a `B i j` query and an `R x l r` query using your design?

ใช้ `vector<set<int>>` เก็บตาราง โดยเก็บแค่ช่องที่ทำการระบายสี `vector<set<int>> rows(n+1)` , `vector<set<int>> cols(n+1)` ; อันนี้ที่นี้แต่ละฟังก์ชันที่มันจะทำงาน

1. เปลี่ยนสีเป็นขาว -> return bool ว่าเจอสีดำ?
2. เปลี่ยนเป็นสีดำ -> return bool ว่าเจอสีขาว? แล้ว add ค่าเข้าไปใน `vector<set<int>>`
3. ตรวจสอบช่อง -> เช็คว่ามีค่า ไยจะใช้ `upper_bound` เช็คว่าเจออยู่ในช่วงมั้ย?
4. ทำคล้ายๆ 3

Let's look more closely at the update operations.

When processing `B i j` (paint cell `(i, j)` black) and `W i j` (paint cell `(i, j)` white), what exact checks and operations do you perform on `rows` and `cols` , and what standard `set` member functions do you call to determine whether to output YES or NO ?

แปลไทยให้หน่อย

Please explain step-by-step for `B i j` and `W i j` :

`cols[j]` ? When do you print **YES** and when **NO** ?

2. For **W i j** : What exact functions do you call on `rows[i]` and `cols[j]` to remove the cell and decide between **YES** and **NO** ?

1. **B i j** -> ใช้ insert ในการใส่ค่าของ row & col เข้าไป (แยก vector) จากนั้น ทำการ find เพื่อหาว่ามี column / row ที่เป็นสีดำอยู่แล้วยั้ย (จริงๆใช้ upper_bound อาจจะดีกว่า) -> print YES เมื่อเจอค่าใน `vector<set<int>>` (ก็คือเจอว่า black อะแหละ)

2. **W i j** -> ใช้ insert ในการใส่ค่าของ row & col เข้าไป (แยก vector) จากนั้น ทำการ find เพื่อหาว่ามี column / row ที่เป็นสีดำอยู่แล้วยั้ย (จริงๆใช้ upper_bound อาจจะดีกว่า) -> print YES เมื่อเจอค่าใน `vector<set<int>>` (ก็คือเจอว่า black อะแหละ)

Let us check the scenario requirements for **W i j** and **B i j** :

For **W i j** (Paint Cell White):

- The requirement says: "Change the color of cell **(i, j)** to **white**. Print **YES** if the cell was **black** right before this query. Otherwise, print **NO** ."

If the cell was black before, what should happen to that cell's coordinates in `rows[i]` and `cols[j]` when painting it white, and what should be output if the cell was already white?

ถ้ามันเป็น black อยู่แล้ว ก็ทำการ erase ออก เพื่อจะได้กลายเป็น white

Understood. And what should you output (**YES** or **NO**) for **W i j** when it was already black vs. when it was already white? And similarly, for **B i j** , when do you output **YES** vs. **NO** ?

W i j -> เช็ค Y/N ก่อน ถ้าเป็น black โห้ YES ถ้าไม่ โห้ NO -> ใน vector ถ้าเจอแล้ว erase ออกจาก vector สีดำ

Now let us look at the range query $R \times l \ r$ (Check Row Range in row x from column l to r).

How do you check whether there is at least one black cell in $rows[x]$ between column l and column r (inclusive) using `set` member functions, without checking every number from l to r one by one?

ถ้าเช็คทุกตัวในช่วง ใช้ `find` แล้วบอกว่าเอา `itr` นี้ถึง นี้ เช่น
`find(black_rows[i]->itr_x, black_rows[i]->itr_y, val_to_find)`
 ประมาณนี้ แต่มันจะกินเวลาค่อนข้างมาก ใช้ `upper_bound` เพื่อหาค่าที่
 มกัลเคียงดีกว่า เพราะมันทำ `binary search` ได้

Let us trace that with a concrete example.

Suppose row x currently has black cells at columns 5 , 20 , and 100 (so $rows[x]$ contains $\{5, 20, 100\}$).

If the query is $R \times 10 \ 50$ (range from column 10 to 50):

1. What member function call on $rows[x]$ would you make (e.g. `lower_bound(10)` or `upper_bound(...)`)?
2. What value does the returned iterator point to?
3. What exact condition do you check on that value to decide whether to output **YES** or **NO** ?

1. เปลี่ยนมาใช้ `lower_bound` แทน เพราะมันค่าที่ $\geq x$ ได้
2. มันจะ return iterator ตัวที่ $\geq x$ ให้
3. เข้าไปเช็คข้างใน row อีกที ว่ามี col ที่ต้องการมั้ย ใช้ `.find` เหมือนเดิม

In your example, $rows[x]$ stores column indices $\{5, 20, 100\}$.

When you call `auto it = rows[x].lower_bound(10);` , `it` points directly to the column 20 .

in the range `[10, 50]` , without calling `.find` again?

เช็ค prev iterator หรือ next iterator เอา

Let us now compare your design with alternatives.

What goes wrong in terms of memory or speed if we instead use:

1. A 2D grid `vector<vector<bool>>` of size `n` by `m` (where `n, m <= 100,000`)?
2. A single `set<pair<int,int>>` storing `(row, col)` for all black cells when answering `C x I r` (column queries)?

1. ต้องไปวน loop ซ้อน loop กิน $O(n^2)$ แน่ๆ อาจติดเรื่อง memory limit
2. `set<pair<int,int>>` ก็ไม่แย่นะ เก็บแค่ row , col ที่ต้องการ แต่มันอาจจะมีปัญหาเวลา erase แล้วทำให้ iterator เพี้ยนได้ (แต่ก็อาจจะมีผลจากการที่ set เก็บ tree ซึ่งกิน time complexity เยอะ)

Thank you for your explanations. That concludes our viva session today.

Grade

Total **60.0 / 100**

Feedback คุณเลือกโครงสร้างข้อมูลพื้นฐานเป็น dual `'vector'` ได้เหมาะสมกับข้อจำกัดของหน่วยความจำ อย่างไรก็ตาม ควรทบทวนการนำ iterator จาก `'lower_bound'` มาใช้ตรวจสอบเงื่อนไขช่วง `'[l, r]'` (เพียงเช็คค่า iterator ไม่เท่ากับ `'end()'` และ `'*it <= r'`) รวมถึงควรทำความเข้าใจเพิ่มเติมเกี่ยวกับการสแกนหาข้อมูลใน single `'set'` เมื่อต้องค้นหาตามแนวคอลัมน์

Rubric container_selection_and_memory

18

Viva Info

Problem	Viva: Cell Detection
User	6730084521
Started	26 minutes ago
Status	submitted
Turns	9
Starts	0 of 1 starts left today

[End interview & get graded](#)[Restart practice viva](#)